

## Tutorial Serie 3

**Objectives:** to master the repetitive instruction (Loops), *while*, *repeat* and *for*.

### Exercise 1

Consider the following algorithms:

```
Algorithm Algo_1 ;
  Var i : integer ;
Begin
  i ← 1;
  While (i ≤ 5) do
    write (i) ;
    i ← i + 1 ;
  EndWhile
End.
```

```
Algorithm Algo_2 ;
  Var i : integer ;
Begin
  i ← 1;
  While (i ≤ 5) do
    i ← i + 1 ;
    write (i) ;
  EndWhile
End.
```

```
Algorithm Algo_3;
  Var i,n, S : integer ;
Begin
  repeat
    read (n);
  Until (n > 0)
  S ← 0;
  i ← n;
  While (i ≥ 1) do
    S ← S + i;
    i ← i - 1 ;
  EndWhile
  write (S) ;
End.
```

```
Algorithm Algo_4;
  Var i,n,Nber,x: integer
Begin
  repeat
    read (n);
  Until (n > 0)
  Nber ← 0;
  i ← 1;
  While (i ≤ n) do
    read (x);
    if (x % 3 = 0) then
      Nber ← Nber + 1;
    Endif;
    i ← i + 1 ;
  EndWhile
  write (Nber) ;
End.
```

- 1- Trace the execution step by step, and describe what does each of above algorithms.
- 2- Choose one of the previous algorithms and rewrite its **While** loop with **Repeat** loop.
- 3- Choose one of the previous algorithms and rewrite its **While** loop with **for** loop.

### Exercise 2

Write an algorithm that calculates and displays the factorial of a positive integer "N".

### Exercise 3

Write an algorithm that calculates and displays  $x^n$  where  $n$  is an integer and  $x$  is a nonzero real number.

### Exercise 4

Write an algorithm that calculates and displays the value of the following expression:

- $\sum_{i=1}^n \frac{x^{i+1}}{(i-1)!}$  //  $n$  is a strictly positive integer, and  $x$  is a nonzero real number.

## Supplementary Exercises

### Exercise 1

Consider the following algorithms:

```
Algorithm Algo_1 ;
  Var i,n : integer ;
Begin
  repeat
    read (n);
  Until (n>0)
  i ← 0;
  While (i ≤ n) do
    write (i) ;
    i ← i+3 ;
  EndWhile
End.
```

```
Algorithm Algo_2;
  Var i,n,S,x : integer
Begin
  repeat
    read (n);
  Until (n>0)
  S ← 0;
  i ← n;
  While (i ≥ 1) do
    read (x);
    S ← S+x;
    i ← i-1 ;
  EndWhile
  write (S) ;
End.
```

```
Algorithm Algo_3 ;
  Var i,n : integer ;
Begin
  repeat
    read (n);
  Until (n>0)
  i ← 0;
  While (i ≤ n) do
    i ← i+3 ;
    write (i) ;
  EndWhile
End.
```

```
Algorithm Algo_4;
  Var i,n,Nber,x,y:integer
Begin
  repeat
    read (n);
  Until (n>0)
  read (x);
  Nber ← 0;
  i ← 1;
  While (i ≤ n) do
    read (y);
    if (y ≥ x) then
      Nber ← Nber +1;
    Endif;
    i ← i+1 ;
  EndWhile
  write (Nber) ;
End.
```

- 1- Trace the execution step by step, and describe what does each of above algorithms.
- 2- Choose one of the previous algorithms and rewrite its **While** loop with **Repeat** loop.
- 3- Choose one of the previous algorithms and rewrite its **While** loop with **for** loop.

### Exercise 2

Write an algorithm that calculates and displays the value of the following expression:

- $\sum_{i=1}^n (i+1)!$  //  $n$  is a strictly positive integer.

### Exercise 3

Write an algorithm to calculate the  $N^{\text{th}}$  term  $U_N$  of the Fibonacci sequence, which is given by the recurrence relation:

- $U_1=1$
- $U_2=1$
- $U_N=U_{N-1} + U_{N-2}$  (for  $N>2$ ).

### Exercise 4

Write an algorithm that displays all divisors of a strictly positive integer « $N$ » read from the keyboard. (Note: display an error message if  $N \leq 0$ ).

### Exercise 5

Write an algorithm that checks if a given positive integer « $N$ » is a **perfect** number or not.

**Definition:** An integer is called **perfect** if it is equal to the sum of its divisors, excluding itself.

**Examples:**  $6 = 1 + 2 + 3$ , so 6 is a perfect number.

$8 \neq 1 + 2 + 4$ , so 8 is not a perfect number.

### Exercise 6

Write an algorithm that checks if a given positive integer « $N$ » is a **prime** number or not.

**Definition:** **prime** numbers are natural numbers that are divisible only by 1 and themselves.

## Exercise 7

---

*Let  $N$  be a strictly positive integer.*

*Write an algorithm to display the sum of the factorials of non-prime numbers less than or equal to  $N$ .*

**Example:** If  $N=10$ , the output should show the calculation:  $R=4!+6!+8!+9!$ .

## Exercise 8

---

*Write an algorithm to find all four-digit integers that are prime numbers and whose digit sum is a prime number.*

**Examples:** Among these numbers are 1013 and 1471.

## Exercise 9: Prime Factorization

---

*Write an algorithm that reads an integer  $N$  greater than 1 and displays its prime factors.*

### Explanation:

To decompose an integer  $N$  ( $N>1$ ) into a product of prime factors, follow these steps:

- Successively divide  $N$  by integers  $P=2,3,4,5,6,\dots$  and check if  $P$  divides  $N$  without a remainder.
- If  $P$  divides  $N$ , display  $P$  and replace  $N$  with  $N/P$ .
- If  $P$  does not divide  $N$ , move to the next integer.
- Stop when  $P$  becomes greater than the square root of the number being decomposed.

### Example:

If  $N=45$ , proceed as follows:

- Divide 45 by 2: 2 does not divide 45 ( $45 = 2 \times 22 + 1$ ) → Move to 3.
- Divide 45 by 3: 3 divides 45 ( $45 = 3 \times 15$ ) → Display 3 and replace 45 with 15.
- Divide 15 by 3: 3 divides 15 ( $15 = 3 \times 5$ ) → Display 3 and replace 15 with 5.
- Since 3 is greater than the square root of 5, stop and display 5.

Thus, the prime factorization of  $N=45$  is  $45=3 \times 3 \times 5$ .

## Exercise 10

---

*Let «  $N$  » be a strictly positive integer.*

*Write an algorithm to determine and display all its digits as well as its length.*

## Exercise 11

---

*Let «  $N$  » be a strictly positive integer.*

*Write an algorithm to determine and display the sum of its digits.*

**Example:** For  $N = 50194$ , the sum of its digits is  $5 + 0 + 1 + 9 + 4 = 19$ .

## Exercise 12

---

*Let «  $N$  » be a strictly positive integer.*

*Write an algorithm to determine and display all its digits that divide it.*

**Example:**  $N = 37142$ ; its digits that divide it are: 7, 1, and 2.

## Exercise 13

---

*Write an algorithm that determines all four-digit integers whose sum of digits is equal to 17.*

## Exercise 14

---

*Write an algorithm that takes an integer «  $N$  », calculates and displays its reverse.*

**Example:**  $N = 107854$ ; its reverse is 458701.

### Exercise 15

---

Write an algorithm that determines and displays all three-digit cubic integers.

A three-digit positive integer is called **cubic** if it equals the sum of the cubes of its three digits.

**Example:** 153 is cubic because  $153 = 1^3 + 5^3 + 3^3$

### Exercise 16

---

Let « S » be a sequence of positive or zero numbers ending with any negative number.

Write an algorithm that checks if « S » is increasing or not.

**Examples:**

The sequence S = 0, 1, 4, 4, 6, 9, 12, 15, 25, 25, 52, -6. is an increasing sequence.

The sequence S = 2, 3, 5, 9, 0, 25, 4, 8, 10, -5. is not an increasing sequence.

### Exercise 17

---

Write an algorithm that reads a decimal number and converts (and displays) it to binary, octal, and hexadecimal.

### Exercise 18

---

Write an algorithm that reads a binary number and displays its decimal conversion.

### Exercise 19

---

Write an algorithm that reads a hexadecimal number and displays its decimal conversion.

### Exercise 20

---

Write an algorithm that calculates and displays the count and percentage of odd numbers within a sequence of positive integers entered by the user, ending with a 0.

### Exercise 21

---

Let the following C program:

```
#include <stdio.h>
main(){
    int n;
    while(1){
        scanf("%d",&n);
        if(n%3 == 0)
            break;
    }
}
```

1. Describe the function of this program.
2. Rewrite the program using a **do...while** loop instead of a **while** loop.
3. Rewrite the program using a **for** loop instead of a while loop.
4. Explain the effect of removing the **break** statement.